



Atividade prática

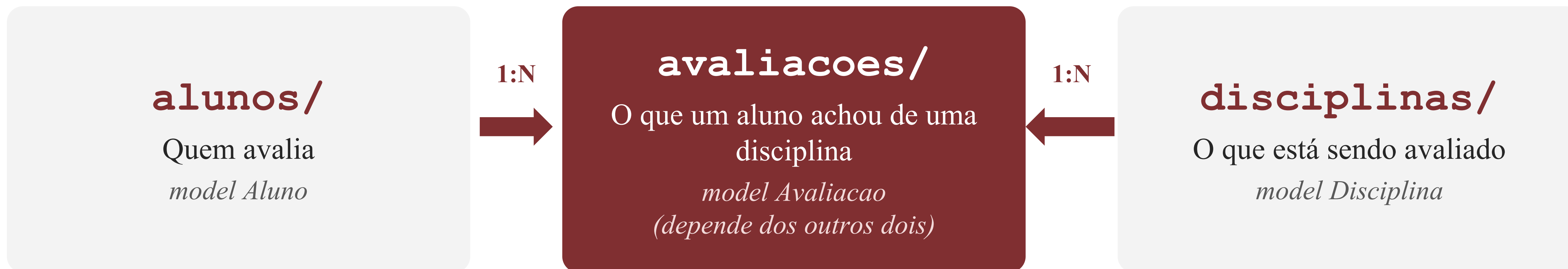
Mini Avaliação Institucional



Lab. Programação Back End

O cenário: um sistema de avaliação

Vocês são a equipe de back end de um sistema em que **alunos avaliam as disciplinas** que cursam. Em um sistema real, os dados de alunos e disciplinas viriam do sistema acadêmico da instituição. Aqui, vocês vão cadastrar esses dados pelo Django Admin e entregar tudo em uma API.



O que vocês vão treinar

Git e GitHub, ambiente virtual e requirements.txt, startproject e startapp, models com ForeignKey, migrations, Admin, views com JsonResponse e rotas com include().

É o mesmo caminho da P1, em outro tema.

Modelo de dados

Aluno

- nome (texto)
- ra (texto, único)
- email (e-mail, único)
- curso (texto)

Disciplina

- nome (texto)
- codigo (texto, único)
- professor (texto)
- periodo (texto, ex.: 2026.2)

Avaliacao

- nota (número inteiro, de 1 a 5, opcional)
- comentario (texto longo, opcional)
- status (PENDENTE ou RESPONDIDA, padrão PENDENTE)
- data_criacao (data e hora automática)
- aluno (ForeignKey para Aluno)
- disciplina (ForeignKey para Disciplina)

Para pensar: por que a ForeignKey fica na Avaliacao e não no Aluno?

Um aluno faz várias avaliações, e uma disciplina recebe várias. Escreva essa justificativa em um comentário no código, ao lado de cada ForeignKey.

O que fazer: preparar e construir

Parte 1: preparar o terreno

1. Criar a pasta avaliacao-institucional e rodar git init
2. Criar o .gitignore e o README.md
3. Dentro de backend/, criar e ativar o venv
4. Instalar o Django e gerar o requirements.txt
5. Criar o projeto com o nome config
6. Fazer o primeiro commit

Parte 2: construir

7. Criar os 3 apps: alunos, disciplinas e avaliacoes
8. Registrar os 3 em INSTALLED_APPS
9. Escrever os 3 models (com as ForeignKeys)
10. Rodar makemigrations e migrate
11. Registrar os models no admin.py e criar o superusuário
12. Cadastrar 3 alunos, 2 disciplinas e 4 avaliações

Lab. Programação Back End

A API: rotas que vocês precisam entregar

Rota	O que devolve	Onde está a dica
/api/alunos/	Todos os alunos	Aula 5: view, urls e include
/api/disciplinas/	Todas as disciplinas	Aula 5: view, urls e include
/api/avaliacoes/	Todas as avaliações	Aula 5: view, urls e include
/api/avaliacoes/pendentes/	Só as avaliações com status PENDENTE	Aula 5: resposta da atividade

Atenção a três detalhes

- Cada app tem o seu urls.py, e o config/urls.py precisa de um include() para cada um.
- Tudo devolve JSON com JsonResponse e safe=False, usando list(queryset.values()).
- Depois de mexer nos models, rode makemigrations e migrate nessa ordem.

Lab. Programação Back End

Git: commits em etapas e entrega

Não deixe tudo para o final: faça commits conforme avança.

1

Commit inicial

após o .gitignore e o README

2

Projeto e apps

após o startproject, os 3 startapp e o requirements.txt

3

Models e banco

após os models, as migrations e o admin

4

API

após as views, as rotas e o teste no navegador

O que NÃO pode subir

venv/, __pycache__/, db.sqlite3 e .env. Confira no GitHub depois do push: se aparecer venv/, o .gitignore está errado.

Entrega

Repositório no GitHub com o nome avaliacao-institucional. Cole o link no AVA.

Lab. Programação Back End

README: explique com as suas palavras

O README.md do projeto precisa ter o passo a passo para rodar e as respostas abaixo, escritas por vocês.

1 Por que usamos um ambiente virtual em cada projeto?

2 Por que dividimos o sistema em 3 apps em vez de um só?

3 Para que servem makemigrations e migrate, e por que nessa ordem?

Respostas copiadas dos slides não valem: expliquem como explicariam para um colega.

Desafio extra

Desafio 1: resumo por disciplina

Crie a rota `/api/avaliacoes/resumo/`, que devolve, para cada disciplina, a média das notas e o total de avaliações RESPONDIDAS.

Dica: pesquise Avg e Count em `django.db.models`, e os métodos `values()` e `annotate()` do ORM.

Desafio 2: troca de computador

Simule trocar de máquina: faça git clone do seu repositório em outra pasta, recrie o venv, instale o `requirements.txt`, rode migrate e suba o servidor.

Se funcionar sem copiar nada na mão, o seu projeto está bem versionado.

Lab. Programação Back End

Checklist antes de entregar

☐

O repositório está no GitHub e o link abre

☐

As migrations estão no repositório

☐

Não aparecem venv/, __pycache__/ nem db.sqlite3 no GitHub

☐

O Admin mostra os 3 models

☐

Existem pelo menos 4 commits, com mensagens que explicam o que mudou

☐

As 4 rotas da API devolvem JSON

☐

O requirements.txt está no repositório

☐

O README tem as 3 respostas escritas com as suas palavras

☐

Os 3 apps estão em INSTALLED_APPS

☐

Cada ForeignKey tem um comentário explicando o motivo